

Riassunto per lo scritto di T.W.

Nicolò Giuliani

1 HTTP

La richiesta via HTTP e' composta:

- *Method*: azione del server che deve eseguire (che il client richiede)
- *URI*: risorsa locale del server
- *Version*
- *Header*: sono i metadati, divisi in **generali** (protocollo, timestamp, ecc), **entita'** (Content-Type, Content-Length, Content-Encoding, ...), **richiesta** (GET/POST)
- *Body*: messaggio MIME (plain text / HTML / Base64 / ...)

Vediamo i metodi di richiesta:

- GET: ottenere una risorsa (link o ricerca nel browser)
- HEAD: uguale a GET ma contiene solo header (senza body), serve solo per la validazione delle risorse
- POST: trasmettere informazioni al server rispetto alla risorsa del URI
- PUT: trasmettere informazioni al server creando o sostituendo la risorsa specificata (inverso del GET)
- DELETE
- PATCH: aggiornare parzialmente una risorsa
- OPTIONS: verificare opzioni, requisiti e servizi

La relativa risposta e' divisa in:

- *Status code* della richiesta
- Version
- Header
- Body: messaggio MIME

2 HTTP/1.1

Prima miglitoria di speedup:

- *pipeline*: invio di piu' richieste senza aspettare risposta, ma le risposte arrivano in ordine delle richieste

3 HTTP/2

Migliorie delle performance tra cui

- *multiplexing* (parallelizzazione delle richieste/risposte ora **compresse** (30% di speedup))
- *push anticipato* del server per il client

4 HTTP/3

Passaggio dal protocollo viaggiante su TCP/IP (HTML 1/2) ad QUIC (basato su UDP).

Dato che HTTP/x e' stateless per il server (non mantiene info delle sessioni precedenti), il client memorizza delle informazioni delle scorse sessioni (**cookies**). Questi sono composti:

- nome
- valori
- *domain*: il dominio nel quale e' valido
- *path*: il percorso del dominio nel quale e' valido
- *expire*: quando non e' piu' valido
- *secure*: se il client deve utilizzare **solo** HTTPS
- *version*

Come si utilizzano?

- *cookies persistenti*: validita' lunga o non scadono, tipo login
- *cookies sessione*: breve durata (cancellati alla chiusura del browser) per raggruppare operazioni in sessioni di lavoro
- *cookies di terze parti*: esterne al dominio della pagina, tipo banner pubblicitari

5 URI

Sono la fusione degli URL (link) e URN (nome univoco come stringa, tipo isbn di un libro).

Gli URI sono progettati per:

- Trascrivibili: hanno un set di caratteri limitato (cosi' possono essere scritti cartelloni, carta, ecc)
- Identificazione: i protocolli non sono definiti dal URI

URI = schema : [//authority] path [?query][#fragment]

Analizziamo:

- Schema: protocollo TCP
- Authority: se presente indica gerarchia dei nomi ([userinfo @] host [: port])
- Path: percorso della risorsa
- Query: ulteriore specificazione della risorsa (generalmente per risposte dinamiche)
- Fragment: individua risorsa secondaria (un frammento) rispetto alla primaria

5.1 Route

Fatta una richiesta di una risorsa il server restituisce in base ad:

- Managed route: hardcoded ogni richiesta da una specifica risorsa (puo'essere anche dinamica)

```
GET(http://www.example.com/name) --> Nicolo'Giuliani
GET(http://www.example.com/css/style.css) --> raw del file css
    /style.css
```

- File-system route: la radice del server corrisponde alla radice dell'URI

```
www/
  index.html      --> http://www.example.com/
  alice/
    index.html    --> http://www.example.com/alice/
  css/
    style.css     --> http://www.example.com/css/style.css
```

5.2 Protocolli morti

- File: file://host/path [#fragment] (dal browser)
- Data: data:[<media type>][;base64],<data> (non cerca ma CONTIENE la risorsa, puo'essere ANCHE codificata in Base64)
- FTP: ftp://[user[:password]@]host[:port]/path

6 API REST

Servizio basato su HTTP e URI come interfaccia per un'applicazione.

Si dice RESTFULL (o anche REST) quando e' composta:

- Route (URI): collezioni o singoli elementi
- metodi HTTP (GET/POST/PUT/DELETE)
- input/output
- errori e relativi messaggi da restituire

La struttura moderna e' il **modello CRUD**:

- Create
- Read
- Update
- Delete

6.1 HTTP Caching

Nelle applicazioni HTTP RESTFULL (API con molte richieste) possiamo avere una cache-server che velocizza la computazione del tempo di risposta delle richieste! NON riduce il traffico sulla rete, per quello c'e'altro.

Per essere sicuri che un oggetto in cache sia ancora valido:

- *Server-specified expiration*: scadenza della risorsa
- *Heuristic expiration*: durata euristica del oggetto in base ad alcune info, tipo "Last-Modified time"

7 Base64

Server per spedire binario via protocolli che vogliono testo (ASCII o UTF-8).

Metodo di codifica e decodifica (algoritmica banale) che permette di codificare a gruppi di 3byte in blocchi da 4 caratteri Base64.

3byte → 4 caratteri Base64 (4byte)
4byte → 8 caratteri Base64 (8byte)
5byte → 8 caratteri Base64 (8byte)
6byte → 8 caratteri Base64 (8byte)

8 JSON Web Token

Token leggero per autenticare client → server.

header.payload.signature

- *header*: tipologia (JWT) e algoritmo (SHA256)
- *payload*: contiene le informazioni di scambio (claims)
- *signature*: garantisce che il token non e'stato modificato, cosi:

base64(header) + "." + base64(payload)

9 CORS

Se in un sito abbiamo un codice maligno che "sniffa" i dati e li rigira ad un altro sito, abbiamo un **cross-domain request**.

CORS e'un meccanismo HTTP che limita le richieste da solo determinati domini esterni! (Funziona per Ajax non per HTML classico).

10 HTML

Protocollo client-server non solo per pagine Web ma anche per appliazioni con richiesta-risposta (API).

10.1 Attributi globali

- *id*: identificati univoco del oggetto
- *class*: la classe del oggetto
- *style*: stile grafico del oggetto
- *title*: info extra per accessibilita'

10.2 Lunghezze

- `width = "150";`
 - valore assoluto di pixel
- `width = "100%";`
 - valore proporzionale al contenitore
- `width = "150, 25%, *, *";`
 - per le tabelle serve per distribuire le varie colonne come:
 - * 150px
 - * 25% della pagina
 - * restante delle pagina / 2
 - * restante delle pagina / 2

11 HTML5

Base del web moderno, con media audio/video nativi, API complesse e nuovi elementi semantici.

- `<video>` / `<audio>`: controlli nativi, piu'sorgenti e sottotitoli. Non servono piu'plugin esterni!
- `<header>` / `<footer>`: Intestazione, menu, logo, titolo, copyright, autori, ecc
- `<article>`: contenuto autonomo (blog)
- `<section>`: contenuti raggruppati per argomento
- `<nav>`: menu, link pagine, breadcrumb
- `<aside>`: extra come sidebar, pubblicita, note
- `<canvas>`: grafica 2D via javascript

Codice piu'leggeibile e **SEO-friendly**.

12 CSS

Sintassi:

```
//per un elemento di HTML
selettore {
    [proprietà]: [statement];
}
//per una classe da noi definita
.classe{

}
//per un oggetto UNICO
#id{

}
//per la prima riga di p
p:first-line{ }

//per solo il testo di p fra "<<...>>"
p{
    color: red;
}
q::before { content : "<<";}
q::after { content: ">>";}
```

HTML prevede l'uso di CSS in 3 modi:

- presso il TAG di riferimento
- nel <style> dell'head
- via file esterno con <link>

E si assegna agli elementi con:

- per tipo (nome elemento)
- per categoria (class)
- specifico elemento (id)

Gli attributi vengono presi a **cascata** ovvero

```
p { font-family: Arial; font-size: 12 pt; }
p { color: red; font-size: 11 pt; }
p { margin-left: 15 pt; color: green;}
```

viene trasformato in

```
p {
    font-family: Arial;
    font-size: 11 pt;
    margin-left: 15 pt;
    color: green;
}
```

12.1 Priorità CSS

- Caso normale:

```
p { color: red; }
p { color: green; }
```

avremo

```
p{
    color: green;
}
```

- Caso **!important**:

```
p { color: red; !important}
p { color: green; }
```

avremo

```
p{
    color: red;
}
```

- In CSS, quando più regole si applicano allo stesso elemento, vince quella con **specificità maggiore**. I selettori sono ordinati per importanza in base a questo principio: selettori più specifici coprono quelli più generali.

<style>

```
nav#main-nav li p { color:red; }
#main-nav .active p { color:yellow; }
body nav#main-nav li p { color:blue; }
nav ul li.active p { color:green; }
```

</style>

<nav id="main-nav">

<li class="active">

<p style="color:black;">link</p>

</nav>

Regola	Specificità	Commento
nav#main-nav li p	0-1-0-3	ID=1, classi=0, tag=3
#main-nav .active p	0-1-1-1	ID=1, classi=1, tag=1
body nav#main-nav li p	0-1-0-4	ID=1, classi=0, tag=4
nav ul li.active p	0-0-1-4	ID=0, classi=1, tag=4
Inline	1-0-0-0	Stile scritto direttamente nell'elemento, specificità massima

→ "color: black"

Tabella 1: Confronto di specificità dei selettori CSS incluso lo stile inline

Riassumendo:

Inline > ID > class > TAG (elemento HTML)

13 Javascript

Durante l'esecuzione dello script, il browser è **bloccato** e non reagisce agli input dell'utente. Come si esegue:

- Sincrono:
 - Appena lo script viene letto da `<script>` o da file
- Asincrono:
 - **event-oriented processing** (ad un evento, es. click bottone)
 - aspettando una operazione di rete
 - dopo un timeout il codice viene eseguito automaticamente

Abbiamo diverse opzioni di output

- `console.log();`
- `alert();`
- `document.write();` (scrive nel documento direttamente)
- `document.getElementById(id).innerHTML = string;` (modificare il DOM visualizzato)

Tipi di dato:

- `bool`
- numeri (`int` e `float`)
- `string`
- `null`
- `undefined`
- **object** (strutture e array)

```
oggetto = {  
    chiave: valore;  
    "altrachiave": altrovalore;  
}
```

→ tutte le chiavi sono trattate come stringhe!

13.1 Accesso alle proprietà di un oggetto in JavaScript

```
const utente = {  
    "profilo": {  
        "info.personali": {  
            "data-nascita": "1990-01-01"  
        }  
    }  
};
```

Possiamo utilizzare la notazione *punto* solo se non abbiamo caratteri come spazi, trattini o punti:

```
utente.profilo
```

Mentre per le chiavi con simboli strani facciamo:

```
utente.profilo["info.personali"];
```

14 Json

JSON (JavaScript Object Notation) è un formato dati derivato dalla notazione usata da JS per gli oggetti.

```
{
  "nome": [ "Giuseppe", "Andrea", "Federico" ],
  "cognome": "Rossi",
  "altezza": 180,
  "nascita": "1995-04-11T22:00:00.000Z",
  "indirizzo": {
    "via": {
      "strada": "Via□Indipendenza",
      "numero": "15"
    },
    "citta": "Bologna",
    "nazione": "Italia"
  },
  "telefono": [
    { "tipo": "casa", "numero": "051□□123456" },
    { "tipo": "cell", "numero": "335□□987654" }
  ]
}
```

Oggigiorno ogni linguaggio di programmazione decente ha un modo per interfacciarsi con i json.

15 Il Document Object Model (DOM)

E'una API per i documenti HTML e XML.

Ogni componente del documento può essere

- letto
- modificato
- cancellato

dal DOM.

JavaScript implementa i metodi standard per accedere al DOM del documento.

```
var c = document.getElementById('c35');
c.setAttribute('class', 'prova1');
c.removeAttribute('align');
var newP = document.createElement('p');
var text = document.createTextNode('Ciao Mamma. ');
newP.appendChild(text);
c.appendChild(newP);
```

15.1 Selettori base HTML in DOM

- `getElementById`
- `getElementsByName`
- `getElementsByTagName`

16 Typescript

E' un super-set di JavaScript, perciò ogni costrutto valido in JavaScript è valido in Typescript. Vengono aggiunti costrutti per l'annotazione dei tipi e un sistema di type-inference che consente di conoscere (staticamente) il tipo di una variabile.

17 Node.js

Node.js è un ambiente di esecuzione Javascript progettato per costruire applicazioni server-side efficienti.

Invece di dedicare un thread ad ogni connessione ricevuta dalla rete, l'applicazione nodeJS utilizza un solo thread per le richieste (asincrone e non bloccanti) e tutte ricevono attenzione condivisa.

- Mantenendo tutto in un unico stack, NodeJS può gestire decine o centinaia di migliaia di richieste concorrenti. (rispetto a poche migliaia)

18 Express.js

Express è un modello di server web per node.js:

- open-source
- molto diffuso, ottima community e plugin

Express si dedica al routing, alla gestione delle sessioni, all'autenticazione degli utenti, e molti altri principi fondanti delle connessioni HTTP.